**SIMATS ENGINEERING**

**ASSESSMENT TOOL 3**

**COURSE CODE: CSA1401** **COURSE NAME: Compiler Design**

**COURSE OUTCOME COVERED**

**CO2: Construct top-down and bottom-up parsers using CFG for parsing conflicts and error**

**recovery for efficient syntax tree generation. (BL3)**

**Assessment Tool Weightage: 10%**

**Duration :60 Mins**

**Off Class**

**QUESTIONS: 3** **Total Marks:30**

**Annexure A**

**SCENARIO BASED TASK**

**Code of Conduct:**

**I Dinesh V (192524309) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.**

**Signature of the student: Dinesh V**

**Scenario 1**

**A parser must detect syntax errors in expressions.**

**Grammar:**

**E → E + T | T**

**T → id**

**Input:**

**id + + id**

**Tasks:**

**(i) Identify error location**

**(ii) Explain error recovery method**

**(iii) Demonstrate parsing steps**

**(iv) Show how parsing continues**

**(v) Suggest improvements**

**Ans:**

**Grammar**

- $E \rightarrow E + T \mid T$
- $T \rightarrow$ id
- **Input:** id + + id

**(i) Error Location**

The error occurs at the **second + token** (Position 3).

- Expecting: an identifier (id) following the operator +.
- Found: an extra binary operator +.

**(ii) Error Recovery Method**

**Panic-Mode / Insertion Recovery Strategy:**

- **Panic-Mode:** The parser discards the unexpected second + until it finds a valid token (id).
- **Insertion / Modification Recovery:** The parser inserts a dummy token (e.g., id or constant 0) between the two plus signs, or deletes the redundant + token to resume parsing.

**(iii) Parsing Steps (Shift-Reduce Simulation)**

Assuming a standard Bottom-Up / LR parser:

| Step | Stack | Input Remaining | Action / Status |
|---|---|---|---|
| 1 | $ | id + + id $ | Shift id |
| 2 | $ id | + + id $ | Reduce $T \rightarrow$ id |
| 3 | $ T | + + id $ | Reduce $E \rightarrow T$ |
| 4 | $ E | + + id $ | Shift + |
| 5 | $ E + | + id $ | **ERROR:** Unexpected token +. Expected id. |

**(iv) How Parsing Continues**

Using **Deletion Recovery** (discarding the duplicate +):

| Step | Stack | Input Remaining | Action / Status |
|---|---|---|---|
| 6 | $ E + | id $ *(after deleting extra +)* | Shift id |
| 7 | $ E + id | $ | Reduce $T \rightarrow$ id |
| 8 | $ E + T | $ | Reduce $E \rightarrow E + T$ |
| 9 | $ E | $ | **ACCEPT** |

**(v) Suggested Improvements**

1. **Extend Grammar for Pre-unary/Post-unary Operators:** Modify the grammar to handle unary operators explicitly ($T \rightarrow +T \mid -T \mid$ id).
2. **Detailed Error Messaging:** Provide precise diagnostic feedback (e.g., "Syntax Error: Unexpected operator '+' at position 3; expected an identifier or expression").

**Scenario 2:**

**A compiler is being designed to parse switch-case statements.**

**Grammar:**

**S → switch ( E ) { C }**

**C → case id : S C | default : S | ε**

**E → id**

**Input:**

**switch ( id ) { case id : id default : id }**

**Tasks:**

**(i) Explain how the grammar handles multiple cases**

**(ii) Demonstrate parsing**

**(iii) Construct parse tree**

**(iv) Identify ambiguity (if any)**

**(v) Suggest improvements**

**Ans:**

**Grammar**

- $S \rightarrow$ switch $(E)\{C\}$
- $C \rightarrow$ case id: $S\ C$ | default: $S$ | $\varepsilon$
- $E \rightarrow$ id
- **Input:** switch ( id ) { case id : id default : id }

**(i) Handling Multiple Cases**

Multiple cases are handled through **right-recursion** on the non-terminal $C$:

- $C \rightarrow$ case id: $S\ C$ allows chaining multiple case blocks sequentially.
- The base case is reached either via default : S or via the empty production $\varepsilon$.

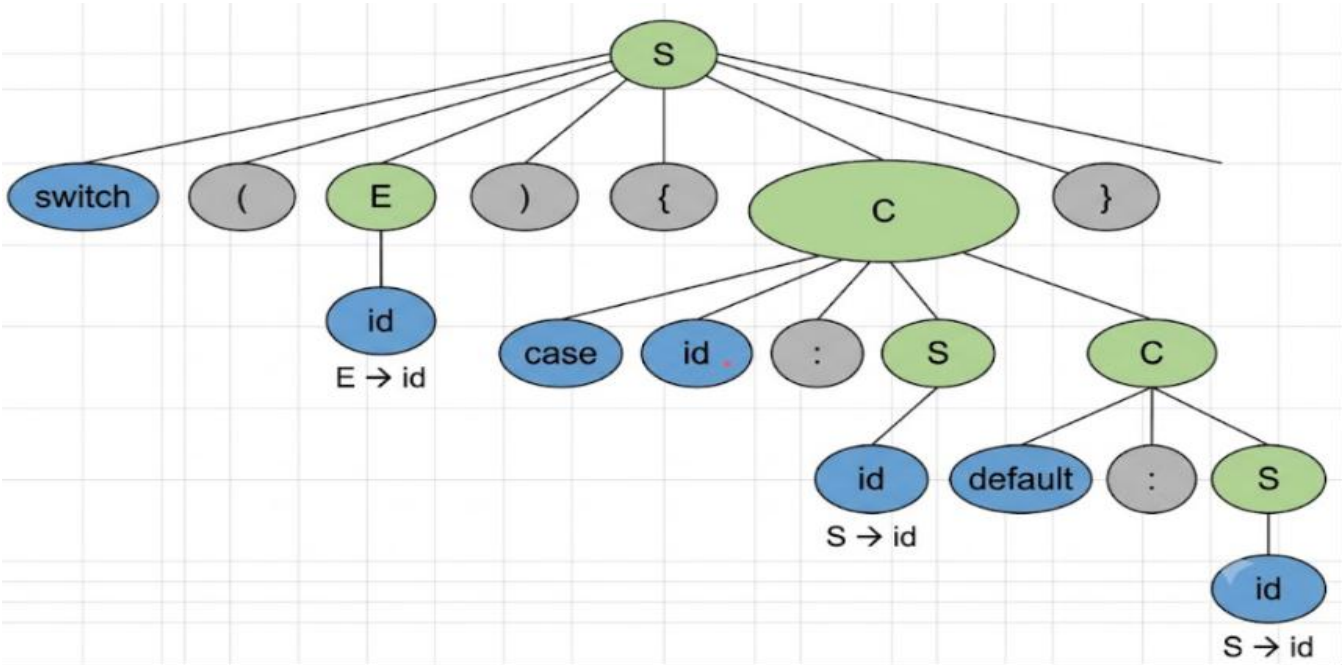
**(ii) Leftmost Derivation**

To parse the input switch ( id ) { case id : id default : id }:

1. $S \Longrightarrow$ switch $(E)\{C\}$
2. $\Longrightarrow$ switch (id)$\{C\}$

3. $\Rightarrow$ switch (id){case id: $S$ $C$}
4. $\Rightarrow$ switch (id){case id:id $C$} *(using statement derivation where* $S \rightarrow$ id*)*
5. $\Rightarrow$ switch (id){case id:id default: $S$}
6. $\Rightarrow$ switch (id){case id:id default:id}

**(iii) Parse Tree Structure**




**(iv) Ambiguity Identification**

- **Dangling Else / Default Ambiguity:** The given grammar allows statements inside a case ($S$) to be arbitrarily structured. If $S$ could produce another switch statement, a default or case clause could ambiguously attach to either the inner or outer switch.
- **Missing Break / Statement Terminators:** In $C \rightarrow$ case id: $S$ $C$, it is ambiguous where statement $S$ ends and where the next case list $C$ begins unless statements have clear end markers (like semicolons).

**(v) Suggested Improvements**

1. **Require Statement Blocks or List of Statements:** Redefining $C \rightarrow$ case id: $S^*$ break; $C$ ensures explicit termination of case bodies.
2. **Explicit Enclosure:** Enforce braces around case blocks to eliminate nested switch-case ambiguities.

**Scenario 3:**

**A parser validates nested HTML-like tags.**

**Grammar:**

**S → < id > S </ id >S | ε**

**Input:**

**< id > < id > </ id > </ id >**

**Tasks:**

**(i) Explain nested structure validation**

**(ii) Demonstrate parsing**

**(iii) Show derivation**

**(iv) Construct parse tree**

**(v) Identify error for invalid nesting**

**Grammar**

- $S \rightarrow \langle \text{id} \rangle\, S\, \langle /\text{id} \rangle\, S \mid \varepsilon$
- **Input:** < id > < id > </id> </id>

**(i) Nested Structure Validation**

The grammar uses **context-free recursion** to enforce balanced matching:

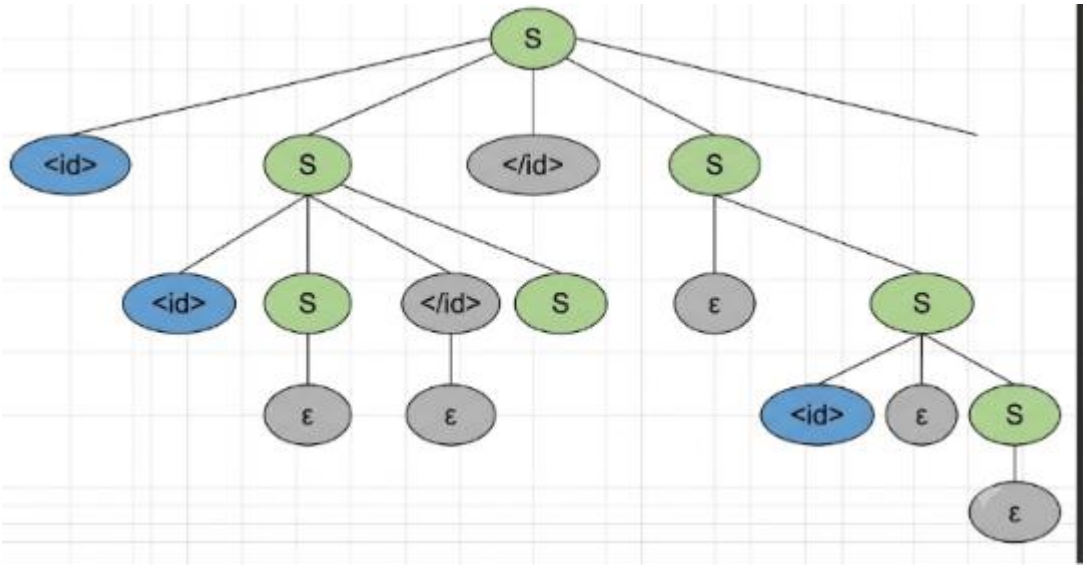
- The production $S \rightarrow \langle \text{id} \rangle\, S\, \langle /\text{id} \rangle\, S$ forces every opening tag $\langle \text{id} \rangle$ to have a corresponding closing tag $\langle /\text{id} \rangle$.
- The inner $S$ permits nested tags between the opening and closing tags.
- The outer (tail) $S$ permits sibling tags following the closing tag.

**(ii) & (iii) Derivation (Leftmost)**

Input tokens: $t_1 = \langle \text{id} \rangle_1, t_2 = \langle \text{id} \rangle_2, t_3 = \langle /\text{id} \rangle_2, t_4 = \langle /\text{id} \rangle_1$

1. $S \Longrightarrow \langle \text{id} \rangle\, S\, \langle /\text{id} \rangle\, S$
2. $\Longrightarrow \langle \text{id} \rangle\, (\langle \text{id} \rangle\, S\, \langle /\text{id} \rangle\, S)\, \langle /\text{id} \rangle\, S$
3. $\Longrightarrow \langle \text{id} \rangle\, \langle \text{id} \rangle\, (\varepsilon)\, \langle /\text{id} \rangle\, S\, \langle /\text{id} \rangle\, S$
4. $\Longrightarrow \langle \text{id} \rangle\, \langle \text{id} \rangle\, \langle /\text{id} \rangle\, (\varepsilon)\, \langle /\text{id} \rangle\, S$
5. $\Longrightarrow \langle \text{id} \rangle\, \langle \text{id} \rangle\, \langle /\text{id} \rangle\, \langle /\text{id} \rangle\, (\varepsilon)$
6. $\Longrightarrow \langle \text{id} \rangle\, \langle \text{id} \rangle\, \langle /\text{id} \rangle\, \langle /\text{id} \rangle$

**(iv) Parse Tree**




**(v) Error Behavior for Invalid Nesting**

For an invalid input like < id > < id > </id> (missing a closing tag):

- **Detection:** The parser expects a closing tag matching the outermost < id >. When the input stream reaches End-Of-File ($), the parse stack still expects </id>.
- **Context-Sensitive Limitation:** Standard Context-Free Grammars (CFGs) can validate structural balancing (like parentheses), but to ensure that the *value* of the opening tag matches the *value* of the closing tag (e.g.,  matched with  instead of ), the compiler must use a **symbol table / semantic action stack** or a **Context-Sensitive Grammar**.